Where Dialog-Only Coordination Breaks: Advisory vs. Enforced Agreement for LLM Agents

Weekly Research Report

Chiharu Tomimori

Institute of Science Tokyo

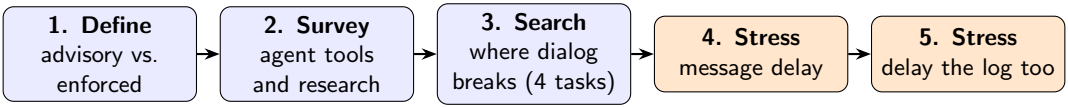
November 6, 2026

Last Week (Oct 30): Feedback and What I Did

Feedback	What I did this week
Accuracy or time?	accuracy is the main metric; time is secondary
Need a baseline	same tool for all; baseline has no rule
Context loss and crash are reversed	fixed: crash loses **new**, context loss loses **old**
"Skill" and "harness" are unclear	new terms: **advisory** and **enforced**
Three-way comparison (none / naive / proposed)	postponed; first find where dialog breaks
Presentation (one message per slide, order, et al.)	applied to all slides

Every number in this talk is a **success** count unless marked.

Progress in This Week: Overview



Main findings

- With slow, asynchronous communication, dialog alone fails; the agreement procedure still succeeds
- Advisory and enforced differ in **how** they fail, not in how often

Background and Motivation

- First 4 tasks this week: baseline succeeded in **20 / 20** runs
  $\Rightarrow$ dialog was enough; no gap to close (ceiling effect)
- **Question**: when does dialog alone break, and does an agreement procedure prevent it?
- **Focus**: not only agreeing, but **acting by the agreement afterwards** (conformance)

Related Work: How Existing Agent Tools Coordinate

Tool	How agents coordinate	Commit a shared choice?
LangGraph [1]	shared state; reducers merge concurrent updates	no (merge)
AutoGen [2]	conversation; a manager picks the next speaker	no (dialog only)
CrewAI [3]	a manager agent delegates	no (manager decides)
OpenAI Agents SDK [4]	handoffs: one agent in control at a time	no (no parallel owners)
A2A protocol [5]	messages and tasks between agents	no (transport only)
Claude Code teams [6]	fixed lead; task list with file lock; mailbox	no (lock for tasks only)

Research: BFT votes on answers [7, 8]; transactions for plans [9]; failures measured, no talk [10]
This work: commit a shared choice, then check that each agent follows it.

[1] LangChain, LangGraph documentation, https://docs.langchain.com/oss/python/langgraph/overview (accessed Nov. 2026).
[2] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," in *Proc. COLM*, 2024.
[3] CrewAI, CrewAI documentation: Processes, https://docs.crewai.com (accessed Nov. 2026).
[4] OpenAI, OpenAI Agents SDK documentation: Handoffs, https://openai.github.io/openai-agents-python/handoffs/ (accessed Nov. 2026).
[5] Agent2Agent (A2A) Protocol specification, https://a2a-protocol.org (accessed Nov. 2026).
[6] Anthropic, "Orchestrate teams of Claude Code sessions," https://code.claude.com/docs/en/agent-teams (accessed Nov. 2026).
[7] L. Zheng et al., "Rethinking the Reliability of Multi-agent System: A Perspective from Byzantine Fault Tolerance," arXiv:2511.10400 (preprint), 2025.
[8] Y. Jo et al., "Byzantine-Robust Decentralized Coordination of LLM Agents," arXiv:2507.14928 (preprint), 2025.
[9] E. Y. Chang et al., "SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning," *PVLDB*, vol. 18, no. 12, 2025.
[10] C. Chacón Sartori, "The Specification Gap: Coordination Failure Under Partial Knowledge in Code Agents," arXiv:2603.24284 (preprint), 2026.

Model: Agents, Channels, Timing

Part	Assumption
Agents	3 LLM agents; each owns one module; no shared files
Channels	message board (broadcast) + shared decision log
Failures	no crash; an agent may **finish and leave early**
Timing	each message is delayed 0–150 s; agents do **not** know the delay
Base method	atomic commit [12]: a decision is final only when **all** participants accept
Tool	`propose`, `respond`, `commit`, `lookup`, `withdraw`, `leave`; append-only log

Timing = partial synchrony [11] (bounded delay, bound unknown).

[11] C. Dwork et al., "Consensus in the Presence of Partial Synchrony," *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.
[12] J. Gray, "Notes on Data Base Operating Systems," in *Operating Systems: An Advanced Course*, LNCS vol. 60, pp. 393–481, Springer, 1978.

Proposed Method (Overview): Advisory vs. Enforced

Rule: "Do not write your file until $\geq$1 decision is committed and none is open. Then follow the decisions, even against your own habits."

	Baseline	Advisory	Enforced
Agreement tool	yes	yes	yes
Rule in the prompt	no	**yes**	no
Hook blocks early writes	no	no	**yes**
Who applies the rule	—	the model	the runtime

Enforced shows the rule text only when a write is blocked.

Task: Three Modules, Three Conflicting Standards

Build an invoice: `pricing.py` (agent 1), `tax.py` (agent 2), `invoice.py` (agent 3)

Standard	Agent 1	Agent 2	Agent 3
Money	float dollars	int cents	Decimal dollars
Percent	fraction (0.10)	number (10)	multiplier (0.90)
Not found	`None`	`KeyError`	default value

- APIs between modules are **not given**: the agents decide them
- Success = all **14 hidden tests** pass after the three files are merged
- Similar designs: information split across agents [13, 14]; conflicting parallel parts [15]

[13] G. Stasser et al., "Pooling of Unshared Information in Group Decision Making," *Journal of Personality and Social Psychology*, vol. 48, no. 6, pp. 1467–1478, 1985.
[14] Y. Li et al., "Systematic Failures in Collective Reasoning under Distributed Information in Multi-Agent LLMs," in *Proc. ICML*, PMLR 306, 2026.
[15] A. Khatua et al., "CooperBench: Why Coding Agents Cannot be Your Teammates Yet," arXiv:2601.13295 (preprint), 2026.

Stress Conditions: Making Communication Slow

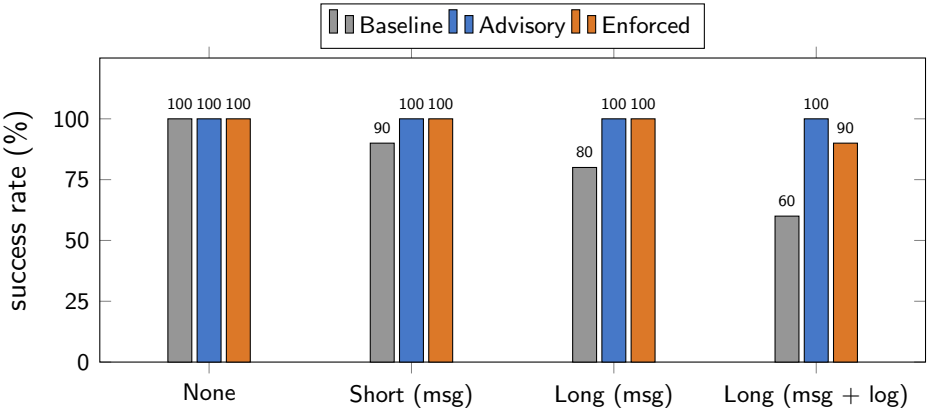
Stress	Message board	Decision log	Runs per cond.
None	no delay	no delay	5
Short delay	20–60 s	no delay	10 / 5 / 5
Long delay	60–150 s	no delay	10
Long delay, both	60–150 s	60–150 s	10

Delay is random per entry; the writer sees its own entry at once. Delayed log: correct but slow.

Environment

- Agent runtime: Claude Code 2.1.318, non-interactive mode (`claude -p`)
- Model: Claude Haiku for all agents
- Limits: 60 turns per agent, 30 min per run
- Machine: cloud container, 2 vCPU (Xeon 2.1 GHz), 7 GB RAM, no GPU
- Software: Python 3.13

Result 1: Success Rate Under Delay



Long, both (10 runs each): baseline 6, advisory 10, enforced 9 successes.

Result 2: Why the Baseline Failed

The 6 baseline failures under long delay share one pattern

Step	What happened	Runs
1	agent 1 or 2 sends its API and stops within 50 s	6 / 6
2	agent 3 writes `invoice.py` **before** those messages arrive	5 / 6
3	agent 3 **guesses** function names	—
4	merge fails: e.g. `pricing` has no attribute `customer_name`	6 / 6

Commit works as a **barrier**: nobody writes until everyone has accepted.

Result 3: Advisory and Enforced Fail Differently

	Baseline failure (4 runs)	Enforced failure (1 run)
What happened	wrote parts that do not fit	wrote nothing; stopped
Property broken	**safety** (wrong result)	**liveness** (no result)
Cause	guessed API under delay	proposals replaced again and again

Enforced run 9: 17 proposals, 14 aborted or withdrawn, 0 commits (float dollars $\leftrightarrow$ int cents).
New versions arrived before votes on old ones were visible $\Rightarrow$ livelock, like duelling proposers in Paxos [16].

[16] L. Lamport, "The Part-Time Parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.

Discussion: What This Week Shows

- Earlier tasks: the baseline never failed $\Rightarrow$ the problem was the **task**, not the method
- This week: built a task where the baseline **does** fail (100 % $\rightarrow$ 60 % success under slow communication)
- The procedure prevents it even with a slow log: the benefit is the **barrier**, not a fast log
- Cost is time: about 2.5$\times$ the baseline with a slow log (696 / 735 s vs. 272 s)
- Advisory agents mostly obeyed the rule: 0 / 10 runs wrote before any commit

Next Week

1. **Remove the confound**: enforced = advisory rule in prompt + hook
2. **Make rule-breaking costly**: a late objection arrives just before commit
   - an agent that writes too early must fail
   - expected: advisory sometimes fails, enforced blocks it
3. Count would-be violations for both conditions with the same metric

Question List

1. Is “bounded but unknown message delay” a convincing model of asynchronous LLM agents?
2. Should liveness failures (no output) count the same as wrong output?
3. Is it fair that enforced hides the rule until a block, or should both conditions see it?
4. Is 10 runs per condition enough for a first paper, or should I aim for 20–30?